Forsíða Viðmótsforritun Hlutapróf 2026

RAFRÆNT HLUTAPRÓF HBV201G VIÐMÓTSFORRITUN

Kennarar: Ebba Þóra Hvannberg (fyrirlestrar)

Deild: Iðnaðarverkfræði, vélaverkfræði og tölvunarfræðideild

Námskeið: HBV201G Viðmótsforritun

ECTS: 8

Vægi hlutapróf : 5% af lokaeinkunn en til hækkunar miðað við lokaprófseinkunn

Próftími: [8:20-9:40]

- **Prófreglur:** Prófið er tekið í Inspera prófakerfinu á tölvur nemenda sem eru stilltar þannig þeim er læst með Safe Exam Browser (SEB).
- **Hjálpargögn:** Engin hjálpargögn
- **Fylgigögn:** Athugið sérstaklega fylgigögn (resources) sem fylgja með prófinu.
- **Fyrirkomulag:** Prófið hefst á krossaspurningum. Þar eru nokkrir svarkostir taldir upp og nemandi velur „réttan eða réttasta“ svarmöguleikann hverju sinni. Ef nemandi á að velja fleiri en einn valkost er það tekið fram. Ekki er dregið niður fyrir röng svör. Í seinni hluta prófsins eru opnar spurningar.
- **Próftaka:** Mikilvægt er að lesa vel spurninguna áður en lokasvar er valið. Eftir að spurningu hefur verið svarað er ýtt á „áfram“ hnappinn (*e. Next*) eða næsta spurning valin á númeruðu flæðilínunni neðst á skjánum. Við aðgerðina vistast svarið sjálfkrafa en einnig vistast vinna nemanda sjálfkrafa á 15-20 sekúndna fresti. Mögulegt er að fara fram og til baka í prófinu og merkja við spurningar sem á eftir að svara.
- **Námsmat:** Einkunn fyrir þetta próf reiknast sjálfkrafa út frá krossaspurningum. Farið er yfir opnar spurningar samkvæmt matskvarða.

Gangi þér vel.

Ebba Þóra Hvannberg

ℹ Hlutapróf 2 Viðmótsforritun Inngangur að hluta 1

Hluti I. Fjölvalsspurningar. Samtals 45%. Opnu spurningarnar í hluta II gilda 55%
Í fjölvalsspurningunum kemur aðeins eitt svar til greina. Ekki er dregið frá fyrir rangt svar.

Ef þið sjáið ekki hluta prófsins, t.d. myndir eða fylgiskjöl, vinsamlegast hafið samband við prófvörð.

Ef galli á prófinu kemur í ljós, verður það skoðað við yfirferð prófsins. Ef nemandi er óviss um hvort skilningur hans á spurningu er réttur, getur hann tilgreint hver sá skilningur er í svarreit við opnum spurningum og/eða sent tölvupóst til kennara eftir prófið.

Eftirfarandi fylgigögn eru með

1 https://prof.snara.is/
2 https://www.bing.com/translator
3 https://openjfx.io/javadoc/25/
4 https://openjfx.io/javadoc/25/javafx.graphics/javafx/scene/doc-files/cssref.html
5 Java cheatsheet

1 Custom component

Hvað lýsir best custom component í JavaFX?

Veldu einn valkost:

- CSS-skrá sem skilgreinir útlit viðmótshluta og gerir þá “custom”.
- Klasi sem erfir alltaf frá Application og býr til nýjan glugga (Stage) fyrir sérhæfðan viðmótshluta.
- Endurnýtanlegur viðmótshlutur sem sameinar nokkra UI-hluti í eina einingu (t.d. spjald með Label og Button) og er hægt að nota aftur í mörgum view-um.
- Sérhæfður vinnsluklasi sem inniheldur reiknireglur og er notaður til að forðast kóða í controller.
- FXML-skrá sem er aðeins hægt að nota einu sinni í forriti, annars myndast villur í SceneBuilder.

Hámarksstig: 5

2 Dialog - resultConverter

Þú ert með Dialog<Vidburdur> sem á að skila nýjum Vidburdur þegar notandi ýtir á OK. Notandi fyllir út gögn og ýtir á OK, en eftir

```
Optional<Vidburdur> result = dialog.showAndWait();
```

er result alltaf tómt (Optional.empty())

Hver er líklegasta ástæðan?

Veldu einn valkost:

- Vidburdur þarf að erfa frá Dialog, annars getur Dialog< Vidburdur > ekki skilað hlut af þeirri gerð (type)
- ButtonType.OK var ekki búinn til með ButtonBar.ButtonData.OK_DONE, þannig að niðurstaðan er ekki túlkuð sem samþykkt
- Dialogurinn þarf að vera stilltur sem non-modal til að skila niðurstöðum
- showAndWait() skilar alltaf tómu Optional nema dialogurinn sé skilgreindur sem Alert
- setResultConverter(...) er ekki sett upp rétt (eða skilar null), þannig að dialogurinn skilar engum Vidburdur þó ýtt sé á OK.

Hámarksstig: 5

3 Dialog með binding

Skoðaðu eftirfarandi kóða

```
public class Person {
   private final StringProperty firstName = new SimpleStringProperty("");
   private final StringProperty lastName  = new SimpleStringProperty("");
   public StringProperty firstNameProperty() { return firstName; }
   public StringProperty lastNameProperty()  { return lastName; }
}
```

Í PersonDialog klasanum eru tvö TextField:

```
public class PersonDialog {

@FXML
private TextField fxFirstName;
@FXML
private TextField fxLastName;

private Person person;

public void setPerson(Person p) {

this.person = p;

fxFirstName.textProperty()
        .bindBidirectional(person.firstNameProperty());

fxLastName.textProperty()
        .bindBidirectional(person.lastNameProperty());
}
}
```

Gert er ráð fyrir að kallað sé á setPerson (Person p) af þeim kóða sem opnar PersonDialog eftir að FXML skráin hefur verið hlaðin inn.

Hvernig er útkoman úr PersonDialog (fornafn og eftirnafn) færð yfir í vinnsluklasann Person ?

Veldu einn valkost:

- ○ með því að kalla á smiðinn í Person
- ○ með validateDialog()
- ○ með setFirstName() og setLastName() í Person
- ○ með setResultConverter
- ○ með bindBidirectional

Hámarksstig: 5

4 Custom component - NamskeidSpjald

Þú ert að búa til custom component NamskeidSpjald sem á að birta upplýsingar um eitt Námskeið (t.d. heiti og ECTS einingar).
Hnapparnir Skoða og Eyða eru í umlykjandi view og hafa sína atburðahandlera í controller.

Hvað er best að láta custom component-inn sjá um?
Veldu einn valkost:

- Að kalla beint á ViewSwitcher þegar notandi velur viðburð
- Að skilgreina CSS-stíla fyrir viðburði svo hægt sé að breyta útliti þeirra
- Að birta gögnin (t.d. með bindingu við Namskeid) og láta umlykjandi controller sjá um atburði og flæði forritsins
- Að búa til og geyma eigið tilvik af vinnsluklasa fyrir hvern viðburð
- Að innihalda alla vinnslurökfræði svo controllerinn þurfi ekki að vita af Namskeid

Hámarksstig: 5

5 Skipta á milli sena

Þú ert með forrit sem skiptir á milli tveggja viðmótstrjáa í sama glugga, t.d. með

- scene.setRoot (nyttRoot) eða
- borderPane.setCenter (nyttRoot) (þar sem nyttRoot er Parent/Node fyrir view)

Þú íhugar að geyma (cache-a) root hlutina svo þú þurfir ekki að hlaða inn nýju tré í hvert skipti

Hver fullyrðing er réttust?

Veldu einn valkost:

- Það er ómögulegt að nota sama root-tré aftur; JavaFX leyfir að root sé aðeins notað einu sinni
- Að cache-a root gerir sjálfkrafa allar bindingar óvirkar þegar skipt er yfir í annað view.
- Cache tryggir að initialize() keyrist aftur í hvert skipti sem skipt er um view
- Cache á root-tré hefur engin áhrif á stöðu viðmótsins; texti og val verða alltaf endurstillt hvort sem er
- Ef þú notar sama root-tréð aftur (cached), þá helst staða view-sins (t.d. texti í TextField og val) oft óbreytt, en ef þú byggir nýtt tré þá byrjar view-ið yfirleitt "upp á nýtt"

Hámarksstig: 5

6 Menu og erfðir

Í JavaFX er hægt að búa til cascading menu (valmynd með undirvalkostum)

Hvernig er þetta útfært með Menu og MenuItem?

Veldu einn valkost:

- Cascading menu myndast sjálfkrafa þegar MenuBar inniheldur fleiri en einn Menu
- Cascading menu er útfært með CSS-stílum sem skilgreina hvernig MenuItem birtast
- MenuItem erfir frá Menu og getur því innihaldið aðra MenuItem, sem gerir cascading menu mögulegt
- Menu erfir frá MenuItem, en getur einnig innihaldið önnur MenuItem, og þannig myndast cascading menu
- Menu og MenuItem eru óskyldir klasar; cascading menu er eingöngu útfært í FXML

Hámarksstig: 5

7 fx.include

Í JavaFX er hægt að nota <fx:include> í FXML til að setja inn annan viðmótshluta

Hver eftirfarandi fullyrðinga lýsir best hlutverki <fx:include>?

Veldu einn valkost:

- < fx: include > er notað til að tengja tvær FXML skrár saman þannig að þær deili sama controller
- < fx: include > afritar Java-kóða úr controller hinnar FXML skrárinnar inn í núverandi controller
- < fx: include > setur viðmótstré úr annarri FXML-skrá inn í núverandi viðmótstré og gerir þannig kleift að brjóta viðmót niður í endurnýtanlega hluta
- < fx: include > virkar aðeins ef viðmótið er opnað í SceneBuilder en ekki þegar FXML er hlaðið í kóða
- < fx: include > tryggir að sama object sé notað í hvert skipti sem vísað er í skrána skránni er

Hámarksstig: 5

8 MediaView og MediaPlayer fullyrðingar

Hvaða fullyrðing er rétt um MediaView og MediaPlayer í JavaFX?

Veldu einn valkost:

- MediaView er UI-hlutur sem teiknar myndband og notar MediaPlayer til að spila miðil.
- MediaView getur spilað myndband án MediaPlayer ef slóðin er rétt.
- MediaView þarf ekki að vera hluti af viðmótstrénu (scene graph) til að myndband birtist á skjánum.
- Media sér um að teikna myndbandið á skjáinn, en MediaView les skrána af diski.
- MediaPlayer er UI-hlutur sem teiknar myndbandið og MediaView spilar hljóðið.

Hámarksstig: 5

9 Scene graph

Í JavaFX er notendaviðmót skipulagt sem tré (scene graph).

Hvaða fullyrðing lýsir viðmótstrénu rétt?

Veldu einn valkost:

- Viðmótstréð er aðeins notað þegar FXML er notað, en ekki þegar viðmótið er búið til í Java-kóða.
- Aðeins container-hlutir eins og VBox og HBox eru hluti af viðmótstrénu, en Label og Button eru ekki nóður.
- Hver nóða getur átt fleiri en eitt foreldri í viðmótstrénu.
- Lauf í viðmótstrénu geta haft mörg börn en foreldranóður ekki
- Allir viðmótshlutir eru nóður (Node) í tré þar sem sumir eru foreldrar (containers) og aðrir eru lauf (leaf nodes) sem innihalda ekki börn

Hámarksstig: 5

Hlutapróf 2 - 2026 Inngangur að hluta II

Hluti II

Í þessum hluta eru 4 spurningar sem gilda samtals 55%.

Gefinn er java ritill til að forrita í. Ritillinn er með setninganúmerum og sýnir java highlights með lit. Ritillinn hjálpar ykkur með réttan inndrátt á línum ef þið notið slaufusviga. Ritillinn athugar ekki hvort forritið er málfræðilega rétt og hann þýðir hvorki né keyrir forritið.

Notið bestu venjur við forritun. Það þarf ekki að setja inn import setningar

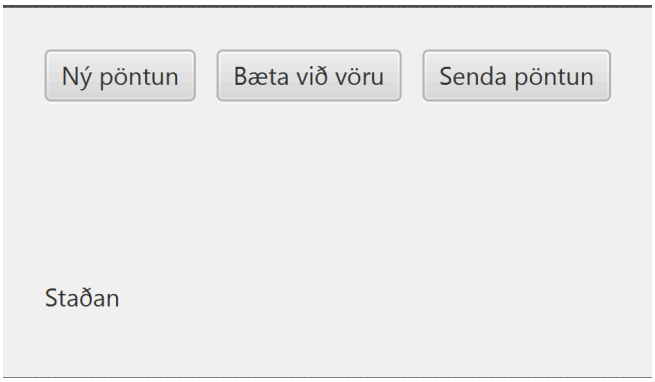
---

Ef þið sjáið ekki hluta prófsins, t.d. myndir eða fylgiskjöl, vinsamlegast hafið samband við prófvörð.

Ef galli á prófinu kemur í ljós, verður það skoðað við yfirferð prófsins. Ef nemandi er óviss um hvort skilningur hans á spurningu er réttur, getur hann tilgreint hver sá skilningur er í svarreit við opnum spurningum og/eða sent tölvupóst til kennara eftir prófið.

```
public class Pontunarkerfi {

    private final ObjectProperty<PontunarStada> stada = new
SimpleObjectProperty<>(PontunarStada.EKKI_HAFIN);

    public ObjectProperty<PontunarStada> stadaProperty() {
        return stada;
    }

    public void nyPontun () {
        stada.set(PontunarStada.I_VINNSLU);
    }

    public void senda () {
        stada.set(PontunarStada.SEND);
    }

}
```

Í einföldu pöntunarkerfi er vinnsluklasi sem heldur utan um stöðu pöntunar. Sjá hér að ofan. Viðmótið hefur þrjá hnappa og eina textabirtingu fyrir stöðu. Hér er skissa af útliti í SceneBuilder



Hér er enum, PontunarStada, fyrir stöðu pöntunar

```
enum PontunarStada {
    EKKI_HAFIN,   // engin pöntun í gangi
    I_VINNSLU,    // verið að setja saman pöntun
    SEND          // pöntun send
}
```

10 Pöntunarkerfi a) - tafla

Eftirfarandi yrðingar segja til um hvort hnapparnir eru virkir eða óvirkir

- Þegar staða er EKKI_HAFIN þá
  fxNyPontun er virkur
  fxBaetaVoru og fxSenda eru óvirkir

- Þegar staða er I_VINNSLU þá
  fxNyPontun er óvirkur
  fxBaetaVoru og fxSenda eru virkir

- Þegar staða er SEND þá
  eru allir hnappar óvirkir og ekki hægt að hefja nýja pöntun

Settu upp töflu þar sem hver hnappur er lína og dálkur er staða og sellan segir til um hvort hnappurinn er virkur/óvirkur

Ritaðu svarið þitt í svargluggann hér að neðan. Öll vinna þín vistast sjálfkrafa.

Stílsnið

Words: 0

Hámarksstig: 5

11 Pöntunarkerfi b) - binding

```
public class PontunarkerfiController {
    @FXML
    private Button fxBaetaVoru;
    @FXML
    private Button fxNyPontun;
    @FXML
    private Button fxSenda;
    @FXML
    private Label fxStadaTexti;
    private Pontunarkerfi kerfi = new Pontunarkerfi();
    public void initialize() {
     // Forritaðu hér
    }
```

Hér fyrir ofan er controller. Þú getur afritað hann í svargluggann.

Skrifaðu kóða í controller, t.d. initialize aðferðina, sem sér um eftirfarandi stillingar á hvort hnappur sé óvirkur (disabled)

- Þegar staða er EKKI_HAFIN þá
  fxNyPontun er virkur
  fxBaetaVoru og fxSenda eru óvirkir
- Þegar staða er I_VINNSLU þá
  fxNyPontun er óvirkur
  fxBaetaVoru og fxSenda eru virkir
- Þegar staða er SEND þá
  eru allir hnappar óvirkir

Þú getur notað annað hvort binding eða listener.

stadaProperty() aðferðin í **Pontunarkerfi** skilar vaktbreytu **(ObjectProperty<PontunarStada>)**

public BooleanProperty disableProperty()

Aðferðin er á Button og skilar BooleanProperty

Athugið að disableProperty() er **true þegar hnappur er óvirkur**.

Ritaðu svarið þitt í svargluggann hér að neðan. Öll vinna þín vistast sjálfkrafa.

```
1
```


Hámarksstig: 10

12 Pöntunarkerfi c) fxStadaTexti

```
public enum PontunarStada {
    EKKI_HAFIN,
    I_VINNSLU,
    SEND
}

public class PontunarkerfiController {
    @FXML
    private Button fxBaetaVoru;
    @FXML
    private Button fxNyPontun;
    @FXML
    private Button fxSenda;
    @FXML
    private Label fxStadaTexti;
    private Pontunarkerfi kerfi = new Pontunarkerfi();
    public void initialize() {
        // forritaðu hér setningar sem beðið er um í þessum lið eingöngu

    }
    // handler þegar ýtt er á nýja pöntun
    @FXML
    void onNyPontun(ActionEvent event) {
        //forritaðu hér
    }
    // handler þegar ýtt er á senda pöntun
    @FXML
    void onSendaPontun(ActionEvent event) {
        // forritaðu hér
    }
}
```

Útfærðu controllerinn þannig að:

1. Þegar ýtt er á fxNyPontun sé kallað á kerfi.nyPontun();
2. Þegar ýtt er á fxSenda sé kallað á kerfi.senda()

3. fxStadaTexti sýnir stöðu pöntunar og uppfærist sjálfkrafa þegar stada breytist.

Þú mátt nota binding eða listener og þú getur valið þér texta sem birtist í fxStadaTexti.

Vísbending (API);

kerfi.stadaProperty() skilar vaktbreytu af taginu **ObjectProperty<PontunarStada>**
og **fxStadaTexti.textProperty()** skilar **StringProperty** sem hægt er að binda við

Athugið að **ObjectProperty** styður **asString()** sem getur verið gagnlegt þegar texti er bundinn við stöðu pöntunar.

Þú getur afritað PontunarkerfiController beinagrind í svargluggann

Ritaðu svarið þitt í svargluggann hér að neðan. Öll vinna þín vistast sjálfkrafa.

Hámarksstig: 15

```
public class Vidburdur {
    private SimpleStringProperty heiti = new SimpleStringProperty();
    private SimpleStringProperty dagsetning = new
SimpleStringProperty();

    public Vidburdur(String h, String d) {
        // kóði er falinn
    }

    public SimpleStringProperty heitiProperty() {
        return heiti;
    }

    public SimpleStringProperty dagsetningProperty() {
        return dagsetning;
    }

    @Override
    public String toString() {
        return heiti.get();
    }
}
```

Viðburðarkerfi er JavaFX forrit sem leyfir notanda að:

- Sjá lista af viðburðum
- Velja viðburð
- Skoða upplýsingar um valinn viðburð

Hver viðburður hefur eiginleikana:

- heiti
- dagsetning

Gefinn er módelklasinn Vidburdur.

Í aðalglugga forritsins er:

- ListView<Vidburdur> sem sýnir nokkra viðburði
- Sérstakt svæði sem á að sýna upplýsingar um valinn viðburð

Til að bæta uppbyggingu forritsins á að útfæra sérstakan **custom component** sem heitir VidburdurSpjald
VidburdurSpjald á að:

- Sýna heiti og dagsetningu eins viðburðar
- Uppfærast sjálfkrafa þegar nýr viðburður er settur í spjaldið

Ekki er gert ráð fyrir View-switching eða mörgum gluggum. Allt er í sama glugga.

13 ListViewVidburðarkerfi liður a)

Útskýrðu hvernig þú myndir hanna VidburdurSpjald sem custom component í JavaFX

Í svarinu skaltu:

- Lýsa hvaða viðmótshluti (t.d. Label, VBox ...) væru í component-inum og hvernig hann væri uppbyggður
- Útskýra hvernig VidburdurSpjald tengist Vidburdur módelinu
- Lýsa hvernig er tryggt að heiti og dagsetning uppfærist sjálfkrafa þegar nýr Vidburdur er settur í spjaldið.

Þetta er ekki kóðaspurning. En svarið þarf að vera nægilega nákvæmt til að hægt sé að kóða út frá því.

Ritaðu svarið þitt í svargluggann hér að neðan. Öll vinna þín vistast sjálfkrafa.

Stílsnið

Words: 0

Hámarksstig: 10

14 ListViewVidburdarkerfi Liður b)

```
public class VidburdurSpjald extends VBox {
    @FXML
    private Label fxHeiti;
    @FXML
    private Label fxDagsetning;

    public  void setVidburdur (Vidburdur v) {
        // forritaðu hér
    }
}

public class VidburdirController {
    @FXML
    private ListView<Vidburdur> fxVidburdir;
    @FXML
    private VidburdurSpjald fxVidburdurSpjald;
    public void initialize() {
        // forritaðu hér
    }
}
```

Skrifaðu Java-kóða sem sér um eftirfarandi:

1. Í VidburdurSpjald skaltu útfæra aðferðina

   public void setVidburdur (Vidburdur v)

   þannig að spjaldið sýni heiti og dagsetningu viðburðarins og uppfærist sjálfkrafa ef v.heitiProperty() eða v.dagsetningProperty() breytast
   Þú mátt nota **binding** eða **listener**

2. Í initialize () í **VidburdirController** skaltu tengja val í ListView við viðburðarspjaldið þannig að kallað sé á **setVidburdur (valinnVidburdur)** þegar val breytist.

   Þú mátt nota **binding** eða **listener**

Ekki er krafist að aftengja (unbind) eldri bindingar

Ritaðu svarið þitt í svargluggann hér að neðan. Öll vinna þín vistast sjálfkrafa.

```
1
```

Hámarksstig: 15